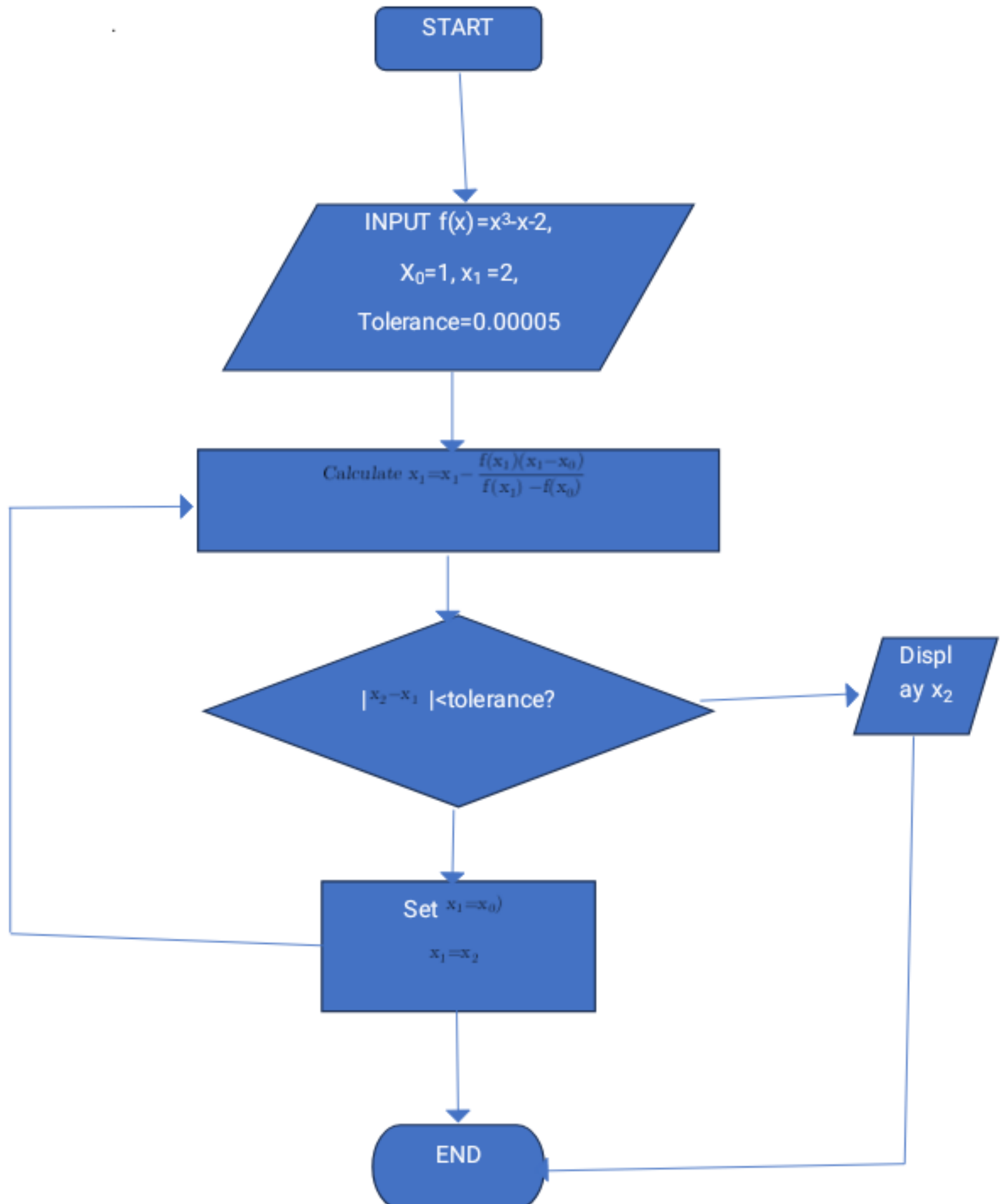


Use the Secant method to find a root of $f(x) = x^3 - x - 2$, using $x_0 = 1$ and $x_1 = 2$. Continue until the root is correct to about 4 decimal places



START

DEFINE $f(x, y) = x + y$

SET $x_0 = 0$

SET $y_0 = 1$

SET $h = 0.1$

SET $x_{\text{end}} = 0.2$

SET $n = (x_{\text{end}} - x_0) / h$ // number of steps = 2

SET $x = x_0$

SET $y = y_0$

FOR $I = 1$ TO n

$K1 = h * f(x, y)$

$K2 = h * f(x + h/2, y + k1/2)$

$K3 = h * f(x + h/2, y + k2/2)$

$K4 = h * f(x + h, y + k3)$

$Y = y + (k1 + 2*k2 + 2*k3 + k4) / 6$

$X = x + h$

PRINT $I, x, k1, k2, k3, k4, y$

END FOR

PRINT " $y(, x_{\text{end}},) \approx , y$ "

END

clc;

clear;

$f = @(x) x.^3 - x - 2;$

$x_0 = 1;$

$x_1 = 2;$

$\text{tol} = 1e-5;$

$N = 20;$

for $k = 1:N$

```

x2 = x1 - f(x1)*(x1-x0)/(f(x1)-f(x0));

fprintf('Iteration %d: x = %.6f\n', k, x2);

if abs(x2-x1) < tol
    x1 = x2;
    break;
end

x0 = x1;
x1 = x2;
end

fprintf('Root = %.6f\n', x1);
f = @(x) x.^3 - x - 2;

x0 = 1;
x1 = 2;
tol = 1e-5;
k = 0;
maxIter = 20;

while k < maxIter
    x2 = x1 - f(x1)*(x1-x0)/(f(x1)-f(x0));
    k = k + 1;

    fprintf('Iteration %d: x = %.6f\n', k, x2);

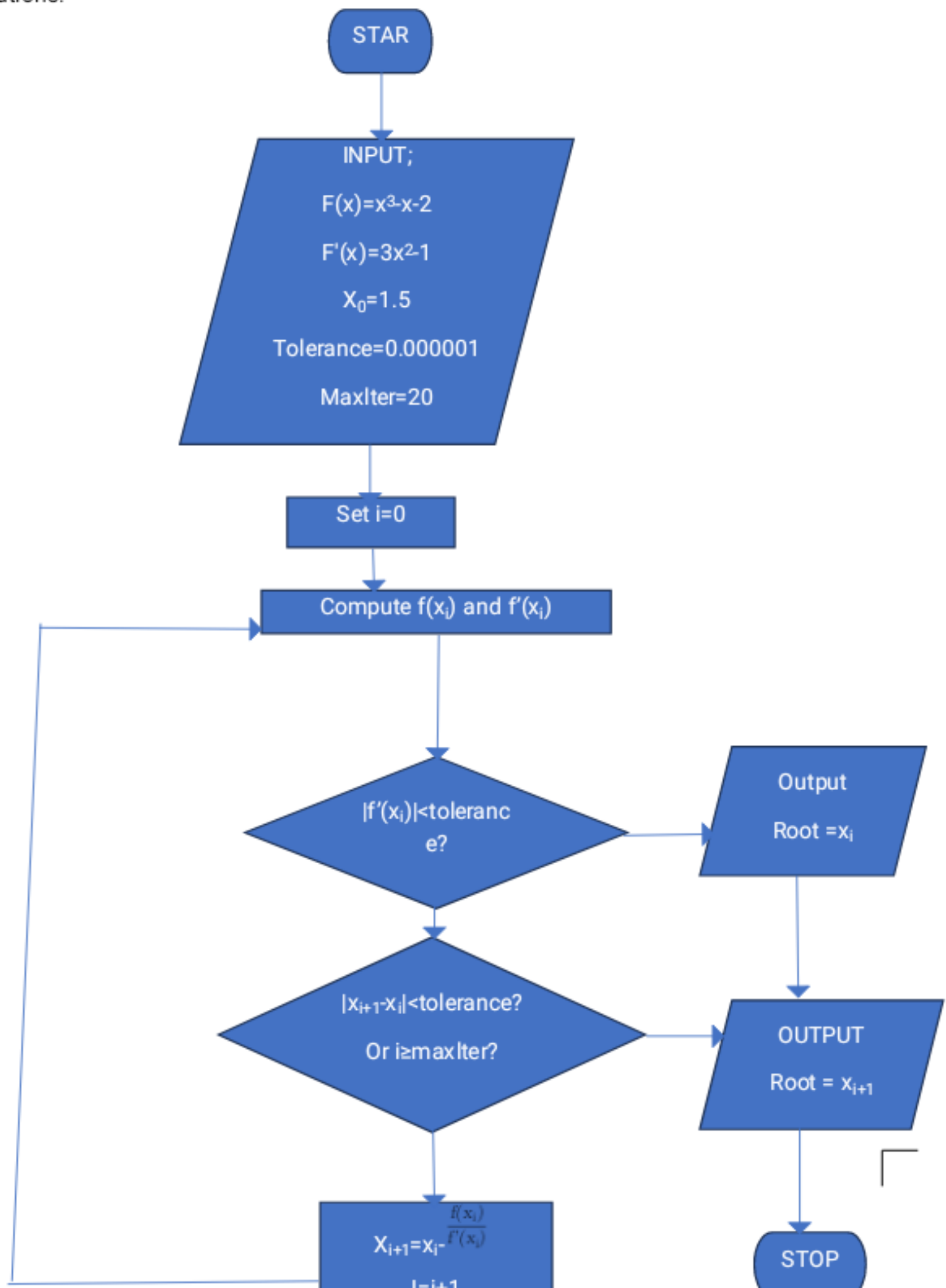
    if abs(x2-x1) < tol
        x1 = x2;
        break;
    end

    x0 = x1;
    x1 = x2;
end

fprintf('Root = %.6f\n', x1);

```

Use the Newton–Raphson method to find a root of the equation $f(x) = x^3 - x - 2 = 0$, starting with the initial guess $x_0 = 1.5$. Use a tolerance of 0.000001 and a maximum of 20 iterations.



PSEUDOCODE

BEGIN

Define $f(x) = x^3 - x - 2$

Define $df(x) = 3x^2 - 1$

Set $x_0 = 1.5$

Set tolerance = 0.000001

Set maxIter = 20

Set $i = 0$

WHILE $i < \text{maxIter}$

$fx = f(x_0)$

$dfx = df(x_0)$

 IF $dfx == 0$ THEN

 Display "Derivative is zero"

 STOP

 END IF

$x_1 = x_0 - fx/dfx$

 error = $ABS(x_1 - x_0)$

 Display $i+1, x_0, x_1, \text{error}$

 IF error < tolerance THEN

 Display "Root =", x_1

 STOP

 END IF

$x_0 = x_1$

$i = i + 1$

END WHILE

Display "Maximum iterations reached"

END

clc;

clear;

format long;

% Newton-Raphson Method

$f = @(x) x^3 - x - 2;$

$df = @(x) 3*x^2 - 1;$

$x_0 = 1.5;$

tol = 1e-6;

maxIter = 20;

fprintf('Iter x_n x_{n+1} Error\n');

```

for i = 1:maxIter
    fx = f(x0);
    dfx = df(x0);

    if dfx == 0
        error('Derivative is zero. Method cannot continue.');
```

end

```

    x1 = x0 - fx/dfx;
    err = abs(x1 - x0);

    fprintf('%2d %14.10f %14.10f %12.10f\n', ...
        i, x0, x1, err);

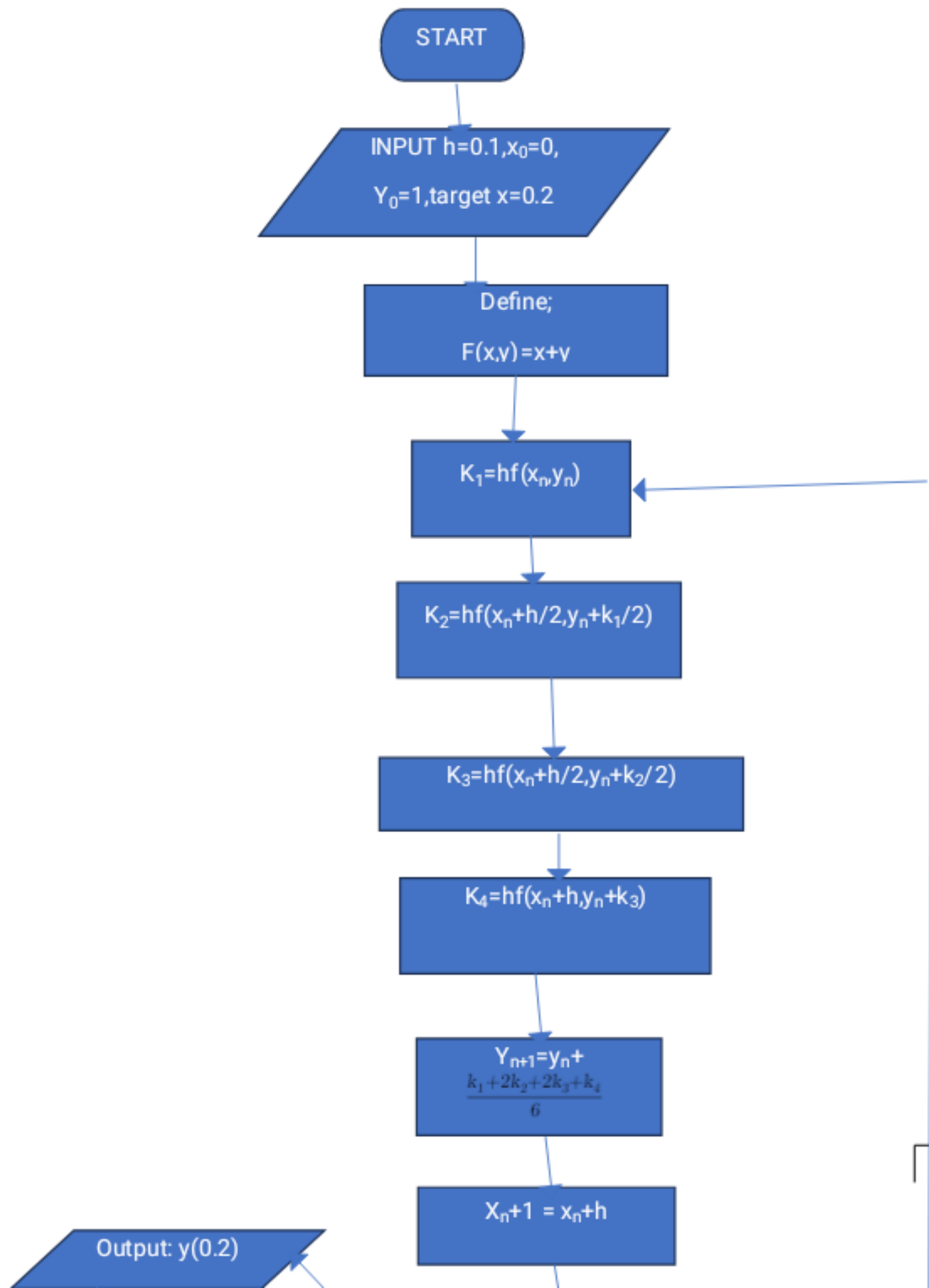
    if err < tol
        fprintf('\nApproximate root = %.10f\n', x1);
        fprintf('Number of iterations = %d\n', i);
        break;
    end

    x0 = x1;
end

if i == maxIter && err >= tol
    fprintf('\nMaximum number of iterations reached.\n');
end

```

Use the fourth-order Runge–Kutta method with $h = 0.1$ to estimate $y(0.2)$, given $dy/dx = x + y$ and $y(0) = 1$.



PESUDOCODE

START

Define $f(x) = x^3 - x - 2$

Set $x_0 = 1$

Set $x_1 = 2$

Set tolerance = 0.00005

REPEAT

Calculate $f_0 = f(x_0)$

Calculate $f_1 = f(x_1)$

Calculate:

$$x_2 = x_1 - [f_1 \times (x_1 - x_0)] / [f_1 - f_0]$$

Calculate $f_2 = f(x_2)$

IF $|x_2 - x_1| < \text{tolerance}$ THEN

PRINT "Root =", x_2

STOP

END IF

Set $x_0 = x_1$

Set $x_1 = x_2$

UNTIL convergence

END


```
clc;
clear;
```

```
f = @(x,y) x + y;
```

```
x = 0;
y = 1;
h = 0.1;
xf = 0.2;
```

```
n = 0;
```

```
while x < xf - 1e-12
```

```
    k1 = h*f(x,y);
    k2 = h*f(x+h/2, y+k1/2);
    k3 = h*f(x+h/2, y+k2/2);
    k4 = h*f(x+h, y+k3);
```

```
    y = y + (k1+2*k2+2*k3+k4)/6;
    x = x + h;
    n = n + 1;
```

```
    fprintf('Step %d: x = %.2f, y = %.6f\n', n, x, y);
end
```

```
fprintf('y(%.2f) = %.6f\n', xf, y);
f = @(x,y) x + y;
```

```
x0 = 0;
y0 = 1;
h = 0.1;
xf = 0.2;
```

```
x = x0;
y = y0;
```

```
N = round((xf-x0)/h);
```

```
for n = 1:N
```

```
    k1 = h*f(x,y);
    k2 = h*f(x+h/2, y+k1/2);
    k3 = h*f(x+h/2, y+k2/2);
    k4 = h*f(x+h, y+k3);
```

```
    y = y + (k1+2*k2+2*k3+k4)/6;
    x = x + h;
```

```
fprintf('Step %d: x = %.2f, y = %.6f\n', n, x, y);  
end
```

```
fprintf('y(%.2f) = %.6f\n', xf, y);
```